

QuerySage Deployment Journey

From Local Development to Google Cloud — A Self-Learning Guide

What This Document Covers

This document captures the complete journey of deploying QuerySage — a multi-agent NL-to-SQL platform — from local development to Google Cloud. It includes every issue encountered, how each was resolved, architecture diagrams, and conceptual explanations of the deployment technologies. Use it as a self-learning reference for deploying ADK 2.0 agents to production.

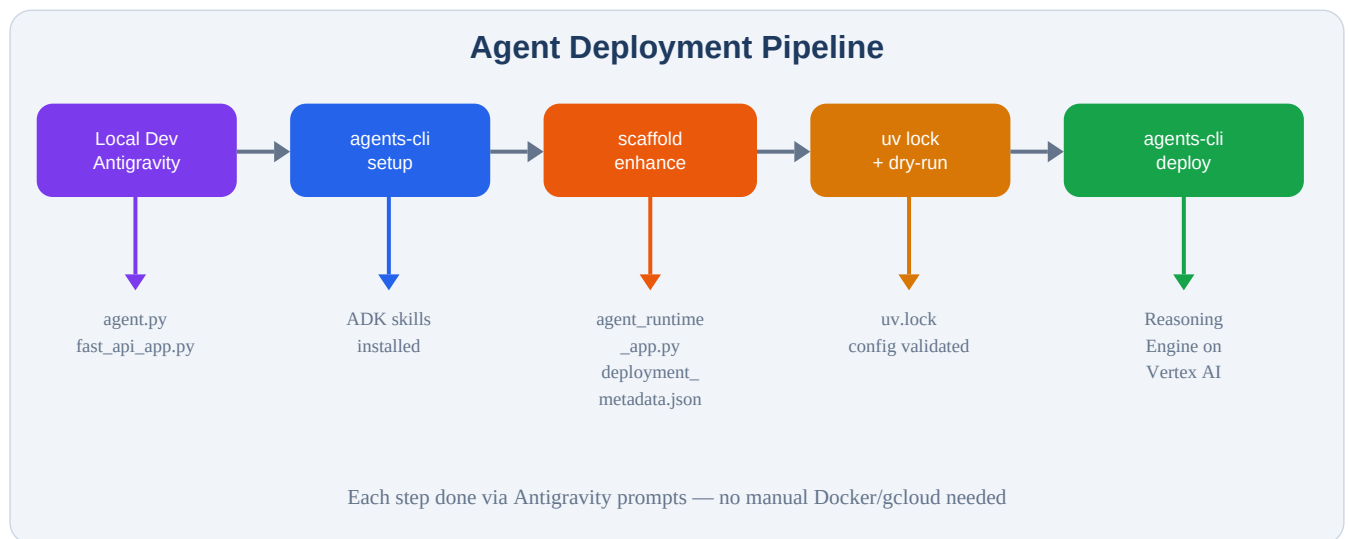


Figure 1 — The 5-step deployment pipeline, each driven by an Antigravity prompt.

Deployment Architecture

QuerySage deploys as 3 **Reasoning Engines** on Agent Runtime (Vertex AI) plus 1 **Cloud Run** service for the Streamlit frontend. Agent Runtime is Google Cloud's managed hosting for ADK agents — it handles scaling, session management, and observability automatically. The Streamlit UI is a standard web app deployed to Cloud Run with a public URL for team access.

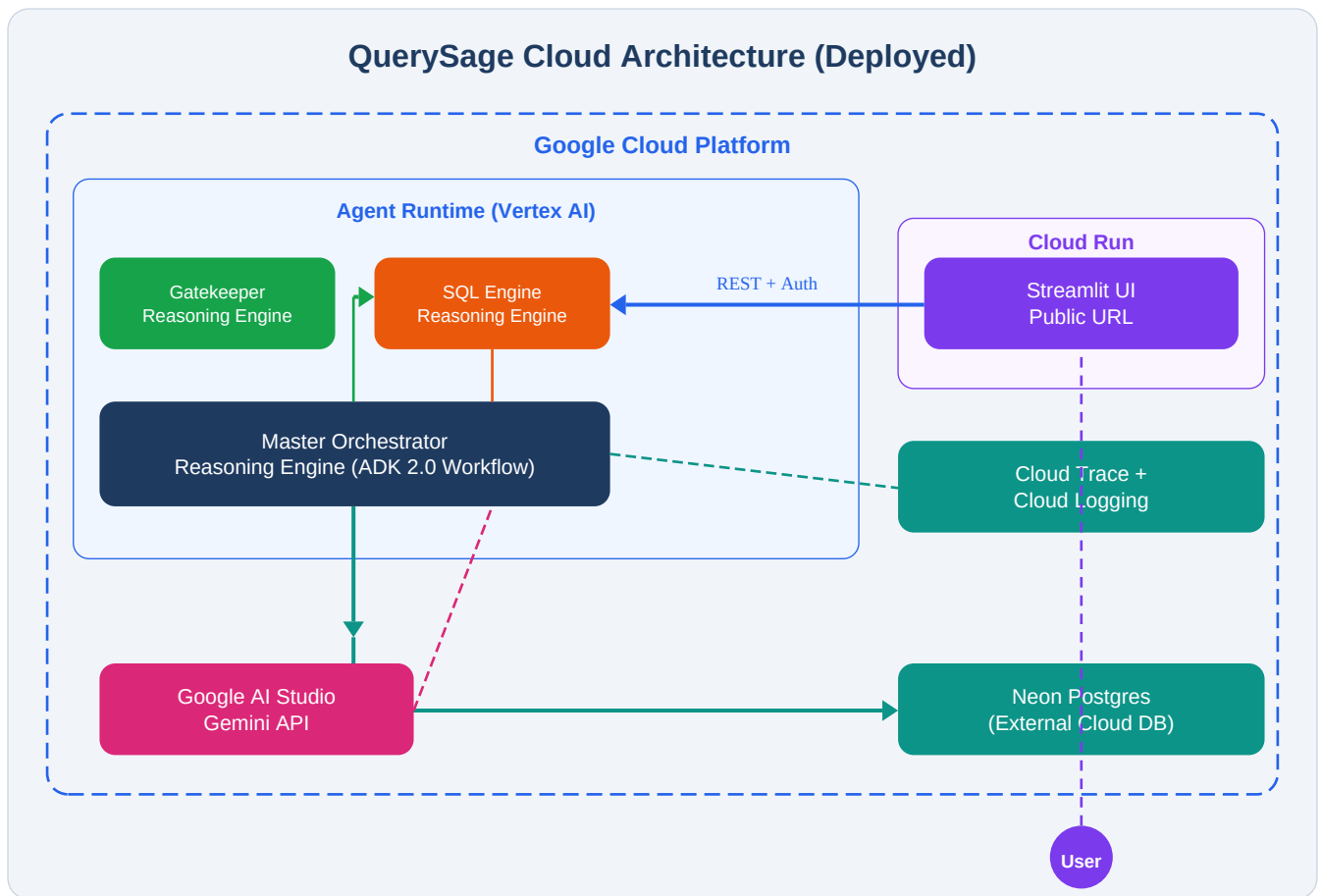


Figure 2 — QuerySage cloud architecture: 3 Reasoning Engines + Cloud Run frontend.

Phase 1: Google Cloud Authentication

Before any deployment, you must authenticate with Google Cloud. This involves two separate auth flows: **gcloud auth login** (for CLI commands) and **gcloud auth application-default login** (for API calls from code/libraries). The second one is often where issues arise.

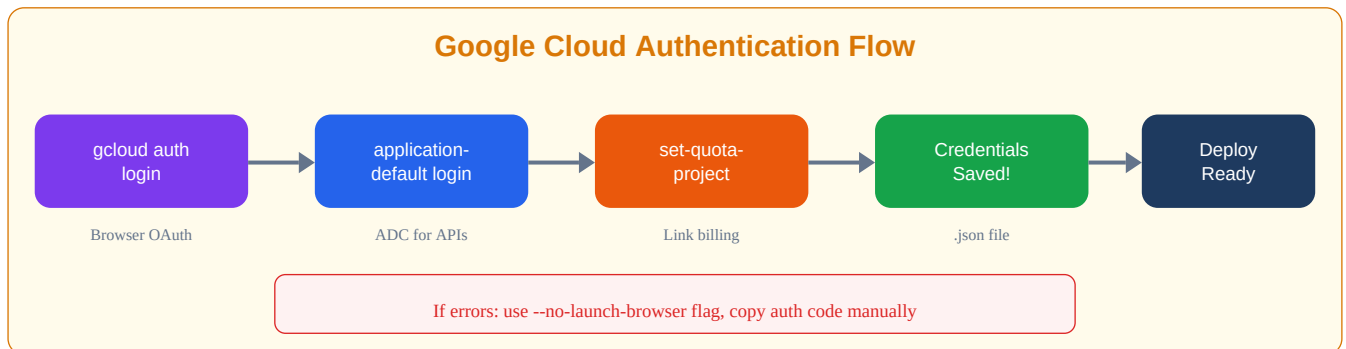


Figure 3 — Authentication flow with fallback path for browser issues.

Issue #1: application-default login Browser Failure

Error: ERROR: There was a problem with web authentication. Try running again with `--no-browser`. scope is required but not consented.

Root Cause: The browser opened but the OAuth consent page wasn't completed — either the page closed too fast, or not all permission scopes were checked.

Fix: Used `--no-launch-browser` flag. This prints a URL to copy-paste into the browser manually, then returns an authorization code to paste back into the terminal.

```
gcloud auth application-default login --no-launch-browser
```

Issue #2: Quota Project Permission Denied

Error: Cannot add the project "quersage-capstone" to ADC as the quota project because the account does not have the "serviceusage.services.use" permission

Root Cause: The Google Cloud project didn't exist yet — the quersage-capstone project ID hadn't been created, so the account had no permissions on it.

Fix: Created the project first, then set the quota project:

```
gcloud projects create quersage-capstone --name="QuerySage"
gcloud config set project quersage-capstone
gcloud auth application-default set-quota-project quersage-capstone
```

Issue #3: IAM Policy Binding Permission Denied

Error: does not have permission to access projects instance [quersage-capstone:getIamPolicy]

Root Cause: Tried to grant IAM roles before the project existed. The `add-iam-policy-binding` command requires the project to already exist and the caller to be an Owner.

Fix: Create the project first (as shown above). The project creator automatically gets Owner role, then all IAM commands work.

Issue #4: Billing Not Linked

Error: Agent Runtime deployment fails because billing is not enabled on the project.

Fix: Opened the GCP Billing Console, created a billing account (or linked existing), and associated it with the project. Google provides free credits for new accounts.

Phase 2: Deploying the Agents

QuerySage has three independent agents that must be deployed in a specific order. The Gatekeeper and SQL Engine have no dependencies, but the Master depends on their deployed URLs. The Streamlit UI depends on the Master's URL.

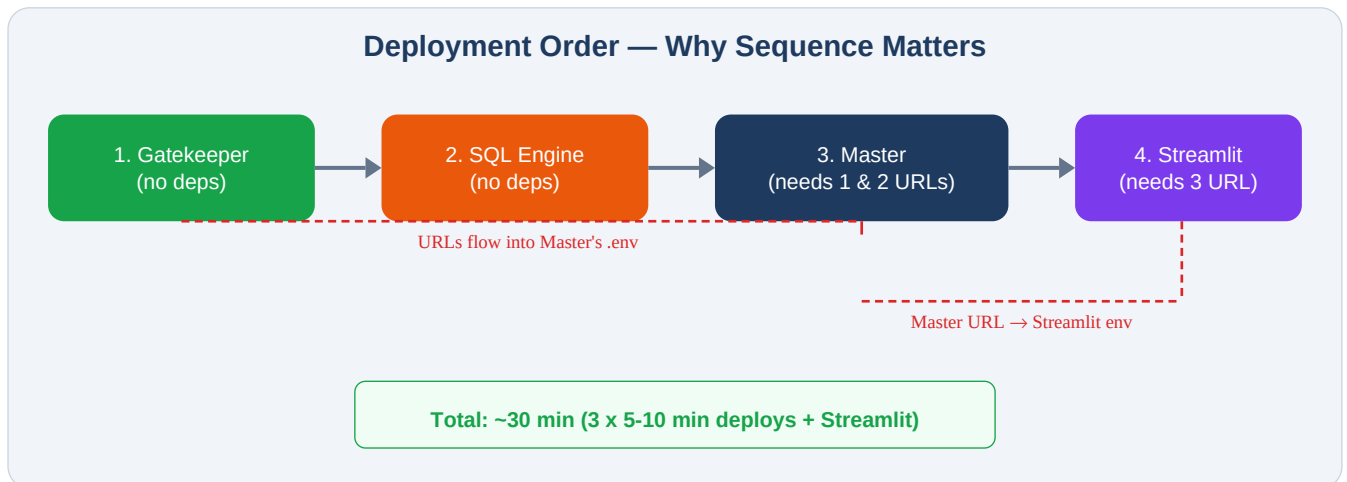


Figure 4 — Deployment order with dependency arrows showing why sequence matters.

Deployment Steps Per Service

Each service follows the same 4-step pattern via Antigravity prompts:

Step	Antigravity Prompt	What It Does
1. Scaffold	"Scaffold production deployment files for Agent Runtime"	Runs agents-cli scaffold enhance Generates agent_runtime_app.py + deployment_metadata.json
2. Lock	"Lock dependencies and run a dry-run deployment"	Runs uv lock + agents-cli deploy --dry-run Validates config without deploying
3. Deploy	"Deploy this agent to Agent Runtime using project X region Y"	Runs agents-cli deploy Packages + uploads + provisions (5-10 min wait)
4. Get URL	"What is the live endpoint URL for my deployed agent?"	Returns the Reasoning Engine API passthrough URL

Issue #5: Inter-Service Authentication

Problem: Locally, the Master calls Gatekeeper and SQL Engine via `http://localhost:9000` — no auth needed. But Agent Runtime endpoints require Google Cloud **Bearer token authentication**.

Solution: Updated `fast_api_app.py` with a `_get_auth_headers()` helper that auto-detects Agent Runtime URLs (containing `aiplatform.googleapis.com`) and adds `Authorization: Bearer <token>` headers. Local `localhost` URLs continue working without auth.

```
async def _get_auth_headers():
    credentials, _ = google.auth.default()
    credentials.refresh(google.auth.transport.requests.Request())
    return {"Authorization": f"Bearer {credentials.token}"}
```

Key Concepts: Agent Runtime & Deployment

Understanding these concepts is essential for deploying and maintaining ADK agents in production. Agent Runtime is Google Cloud's managed hosting specifically designed for AI agents — it's different from generic Cloud Run or Cloud Functions.



Figure 5 — Agent Runtime ecosystem: the 6 key services that support your deployed agent.

Concept	What It Is	Why It Matters
Reasoning Engine	A managed Vertex AI resource that hosts your ADK agent	Always-on endpoint — doesn't stop when you close your laptop
agents-cli	CLI tool that scaffolds, validates, and deploys ADK agents	Automates Docker build, push, and Agent Runtime provisioning
scaffold enhance	Generates agent_runtime_app.py and deployment_metadata.json	Production wrapper that Agent Runtime needs to host your agent
dry-run	Validates config + deps without actually deploying	Catches errors before spending 5-10 min on a real deploy
Cloud Trace	Distributed tracing for every agent interaction	See execution spans for each workflow node + latencies
ADC (Application Default Credentials)	JSON file with OAuth tokens for API authentication	Required for inter-service calls in the cloud environment

Local vs Cloud: What Changes

The core agent logic (`agent.py`) stays identical. What changes is the hosting wrapper and how services discover each other. Locally, everything runs on `localhost` with no auth. In the cloud, each service gets a Vertex AI URL and requires Bearer token auth.

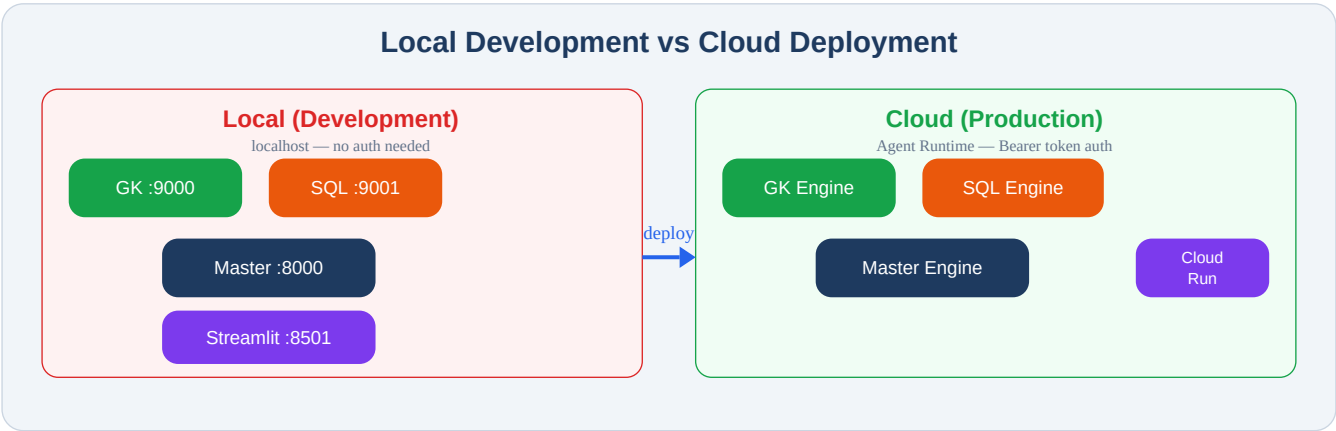


Figure 6 — Side-by-side comparison of local development vs cloud deployment.

Aspect	Local	Cloud
Service discovery	localhost:PORT	Reasoning Engine API URL
Authentication	None	Bearer token (google.auth)
Session storage	InMemorySessionService	Agent Runtime managed sessions
Monitoring	Console logs	Cloud Trace + Cloud Logging
Availability	Stops when laptop closes	Always-on 24/7
Scaling	Single process	Auto-scales with demand
Frontend	streamlit run (local)	Cloud Run public URL
Cost	Free	Free tier (demo usage)

Streamlit Frontend on Cloud Run

The Streamlit app deploys to **Cloud Run** (not Agent Runtime) because it's a frontend web app, not an ADK agent. Cloud Run gives it a **public URL** (`https://quersage-ui-xxxxx-as.a.run.app`) that anyone can open in their browser. The app uses environment variables (`MASTER_URL`) to point to the Master's Agent Runtime endpoint, and includes `google-auth` to add Bearer tokens when calling the cloud-hosted Master.

Complete Deployment Checklist

#	Task	Command / Action	Status
1	gcloud auth login	gcloud auth login	Done
2	application-default login	gcloud auth application-default login --no-launch-browser	Done
3	Create GCP project	gcloud projects create queriesage-capstone	Done
4	Enable billing	GCP Console > Billing > Link account	Done
5	Enable APIs	gcloud services enable aiplatform.googleapis.com cloudbuild.googleapis.com cloudtrace.googleapis.com	Done
6	Install agents-cli	uvx google-agents-cli setup	Done
7	Deploy Gatekeeper	Antigravity: scaffold + deploy	Done
8	Deploy SQL Engine	Antigravity: scaffold + deploy	Done
9	Update Master .env	Set GATEKEEPER_SERVICE_URL and SQL_ENGINE_SERVICE_URL to Reasoning Engine URLs	Done
10	Deploy Master	Antigravity: scaffold + deploy	Done
11	Deploy Streamlit	Antigravity: deploy to Cloud Run with MASTER_URL env var	Done
12	Test end-to-end	Open public Streamlit URL, submit query, approve SQL	Done
13	Git commit & push	git add -A && git commit && git push	Pending

Issues Summary

#	Issue	Root Cause	Resolution
1	Browser auth failure	OAuth consent not completed	Used --no-launch-browser flag
2	Quota project denied	Project didn't exist yet	Created project first, then set quota
3	IAM binding denied	Project didn't exist	Create project before granting roles
4	Billing not linked	New project has no billing	Linked via GCP Billing Console
5	Inter-service auth	Agent Runtime needs Bearer tokens, localhost doesn't	Added _get_auth_headers() that auto-detects cloud URLs